

Project Overview: Automated Index Generation

The goal was to modify the PostgreSQL executor to track sequential scans on table columns and automatically trigger a CREATE INDEX command once a specific frequency threshold is reached.

Changes Made to the Codebase

1. `src/include/utils/autoindex.h`

Changes:

- Defined the `AutoIndexKey` struct (mapping relid and attnum).
- Defined the `AutoIndexEntry` struct (storing the scan_count).
- Set the `AUTOINDEX_THRESHOLD` (the trigger limit, currently set to 5).
- Declared function prototypes for shared memory initialization and tracking.

Reasoning:

Centralizing these definitions ensures that both the shared memory management and the executor-level tracking use the same data structures and threshold logic.

2. `src/backend/storage/ipc/ipci.c`

Changes:

- Modified `CreateSharedMemoryAndSemaphores` to include calls to `AutoIndexShmemSize()` and `AutoIndexShmemInit()`.

Reasoning:

PostgreSQL uses a fixed-size shared memory segment. We must register our new hash table during the server's pre-initialization phase so that all background processes (backends) share the same scan counters.

3. src/backend/executor/nodeSeqscan.c

Changes:

- Added a hook inside ExecInitSeqScan.
- The code iterates through the scan's filter predicates (plan->qual) to identify equality expressions (e.g., WHERE col = 100).
- Extracts the AttrNumber and calls TrackEqualityPredicate().

Reasoning:

ExecInitSeqScan is the "entry point" for every sequential scan. By intercepting the plan here, we can accurately record when a developer is querying an unindexed column repeatedly.

4. src/backend/utils/misc/autoindex.c

This is the core logic file. Significant refinements were made here to handle PostgreSQL's strict memory and transaction safety rules.

Key Logic: TrackEqualityPredicate

- **System Filter:** Added an OID check (`relid < 16384`) to ignore System Catalogs.
“ Why: This prevents the recursive loop where the auto-indexer tries to index Postgres's own metadata tables during a `\d` command.
- **Shared Memory Access:** Uses `LWLockAcquire (AddinShmemInitLock)` to ensure thread-safe increments of scan counts across multiple concurrent users. “

Key Logic: ExecuteAutoIndex

- **Variable Scoping:** All declarations were moved to the top of the function to comply with ISO C89 standards.
“
- **Memory Management:** Added `pstrdup()` for relation and attribute names.
Why: This copies strings into local memory, preventing "use-after-free" errors if the system cache is invalidated during index creation.
- **Manual Node Construction:** Instead of using the version-dependent `makeIndexElem()`, we manually initialized the `IndexElem` struct.
Why: This bypassed "undeclared function" errors caused by refactored headers in the PostgreSQL source.

- **Transaction Safety (PG_TRY / PG_CATCH):** Wrapped the DefineIndex call in a `try` block.
Why: This allows the original `SELECT` query to finish even if the index creation hits a lock conflict, preventing a full session crash.
- **Concurrency Setting:** Set `stmt->concurrent = false`.
Why: While concurrent builds are great for production, they cannot be triggered from inside an active transaction. Standard builds are significantly more stable for this specific executor-hook implementation.

Final Results

The system now successfully:

- Detects repeated scans on user tables.
- Ignores internal system queries.
- Automatically generates valid B-Tree indices.
- Maintains database stability by catching resource-owner conflicts.

Current Implementation Status

- **Success:** Indices are appearing in `\d` after reaching the threshold.

Future Extensions

1. Cost-Benefit Thresholding

- **Concept:** Replace the "magic number" threshold with a financial model of system resources.
- **Logic:** Trigger creation only when $(N \times \text{Savings}) > \text{Creation Cost}$ where `Savings = cost_seqscan - cost_indexscan`
- **Implementation:** Utilize internal `cost_seqscan` and `cost_indexscan` metrics to estimate the "break-even" point for CPU and I/O.

2. Asynchronous Background Worker (BGW)

- **Concept:** Decouple index creation from the user's `SELECT` process.
- **Logic:** The Executor places a request in a shared memory queue; a dedicated BGW process handles the DDL.

- **Benefit:** Eliminates "Resource Owner" conflicts and session latency. Enables the safe use of `CREATE INDEX CONCURRENTLY`.

3. Selectivity Analysis

- **Concept:** Prevent "weak" indexes on low-cardinality columns (e.g., Booleans, Gender).
- **Logic:** Query `pg_statistic` to check the `stadistinct` value.
- **Action:** Abort auto-indexing if the column filters out less than a significant percentage (e.g., <20%) of the total table rows.

4. Composite (Multi-Column) Indexing

- **Concept:** Recognize patterns of multi-clause queries.
- **Logic:** If `WHERE A = x AND B = y` is common, detect the co-occurrence of predicates in the plan tree.
- **Outcome:** Generate a single composite index (A, B) instead of two redundant single-column indexes.

5. "Human-in-the-Loop" Approval Mode

- **Concept:** Provide administrative oversight for database changes.
- **Logic:** Instead of immediate execution, log proposed indexes to a system table (e.g., `pg_index_proposals`).
- **Outcome:** Allows DBAs to review, test, and "approve" auto-generated suggestions before they impact production storage.